

8085 Assembly Language Programming — Solutions

Course: Computer Architecture (CSA12) | CO1 — Analyse the functional units of a computer system | Assessment Tool 3

Each problem below is solved with a fully-commented 8085 program together with its algorithm, the memory map for data and results, a worked sample test case (input and the exact expected output for verification), and an explanation of the registers, flags and logic used. Program code and data occupy **separate memory regions**: the data/result block shown in every memory map lies in RAM starting at 2000H, while the program itself may be assembled at any convenient free area (for example 8000H). All quantities are treated as **8-bit unsigned** numbers, and 16-bit accumulation is used wherever a sum can exceed FFH (255).

Flags used throughout: **CY** (carry/borrow) and **Z** (zero), both set by the `ADD` and `CMP` instructions. `CMP X` performs `A - X` without changing A: it sets CY when `A < X` and Z when `A = X`.

The hourly production counts of eight machines are stored in consecutive locations. Compute the total production, find the highest producer, compare the total against the stored daily target, and store a status flag showing whether the target has been achieved (`total ≥ target`).

Algorithm

- Initialise: pointer → first reading, counter = 8, 16-bit total = 0, highest = 0.
- Read a value; add it to the 16-bit total, incrementing the high byte whenever `ADD` produces a carry.
- Compare the value with the current highest; if it is larger, make it the new highest.
- Advance the pointer, decrement the counter, and repeat until all eight are processed.
- Store the total (low, high) and the highest value.
- Compare the 16-bit total with the 16-bit target — high bytes first, then low bytes.
- Store 01H if `total ≥ target`, otherwise 00H.

Memory Map

2000H – 2007H	Production of machines 1–8 (input)	0A 14 1E 28 32 3C 46 50
2008H / 2009H	Daily target low / high byte (input)	2C / 01
200AH / 200BH	Total production low / high byte (output)	68 / 01
200CH	Highest production value (output)	50
200DH	Status flag 01 = achieved, 00 = not (output)	01

8085 Assembly Program


```

;=====
; Q1 : SMART FACTORY - PRODUCTION ANALYSIS
;=====
; DATA
; 2000H - 2007H : production count of 8 machines
; 2008H : daily target - LOW byte
; 2009H : daily target - HIGH byte
; RESULTS
; 200AH : total production - LOW byte
; 200BH : total production - HIGH byte
; 200CH : highest production value
; 200DH : status flag (01=achieved, 00=not)
;-----
LXI H,2000H ; HL -> first machine reading
MVI C,08H ; C = machine counter (8)
MVI B,00H ; B = total HIGH byte = 0
MVI E,00H ; E = total LOW byte = 0
MVI D,00H ; D = highest value = 0

LOOP: MOV A,M ; A = current production
ADD E ; add to running total (low)
MOV E,A ; save low byte
JNC SKIP ; no carry -> high byte same
INR B ; else carry into high byte
SKIP: MOV A,M ; reload current production
CMP D ; compare with highest so far
JC NEXT ; current < highest -> skip
MOV D,A ; else highest = current
NEXT: INX H ; point to next machine
DCR C ; one machine done
JNZ LOOP ; repeat for all 8

; ---- store total and highest ----
MOV A,E
STA 200AH ; total LOW byte
MOV A,B
STA 200BH ; total HIGH byte
MOV A,D
STA 200CH ; highest production value

; ---- compare total(B:E) with target(2009H:2008H) ----
LDA 2009H ; A = target HIGH byte
MOV C,A ; C = target HIGH
MOV A,B ; A = total HIGH
CMP C ; total high - target high
JC NOTMET ; total < target -> not met
JNZ MET ; total > target -> met
LDA 2008H ; equal high: now test low
MOV C,A ; C = target LOW
MOV A,E ; A = total LOW
CMP C ; total low - target low
JC NOTMET ; total < target -> not met

MET: MVI A,01H ; target achieved
STA 200DH
JMP DONE
NOTMET: MVI A,00H ; target NOT achieved
STA 200DH
DONE: HLT ; stop

```

Sample Test Case (Verification)

Inputs: machines = 10, 20, 30, 40, 50, 60, 70, 80 (0A–50H); target = 300 (012CH, stored as 2C at 2008H and 01 at 2009H).

200AH : 200BH	68 : 01	Total = 0168H = 360
200CH	50	Highest = 80
200DH	01	360 ≥ 300 → target ACHIEVED

Explanation

HL is the data pointer that walks through the eight values; **C** is the loop counter (`DCR C / JNZ` repeat the body eight times). Because eight readings can sum beyond 255, the total is kept as 16 bits in the register pair **B** (high) and **E** (low); after every `ADD E` the **CY** flag is tested and **B** is bumped on overflow. **D** holds the running maximum: `CMP D` computes value-D, and a cleared carry (value $\geq$ D) updates it. Finally a true 16-bit magnitude compare (high bytes, then low bytes, using CY and Z) decides 'achieved', and 01H or 00H is written to 200DH.

Ten temperature readings are scanned. A reading is abnormal when it is strictly greater than the stored threshold. The program counts the abnormal readings and records the maximum temperature, storing both results in memory.

Algorithm

- Initialise: pointer → first reading, counter = 10, abnormal count = 0, maximum = 0; load the threshold into E.
- Read a value; if it is greater than the threshold, increment the abnormal count.
- Compare the value with the current maximum; if it is larger, update the maximum.
- Advance the pointer, decrement the counter, and repeat until all ten are processed.
- Store the abnormal count and the maximum temperature.

Memory Map

2000H – 2009H	Ten temperature readings (input)	19 1E 2D 32 3C 23 46 14 37 41
200AH	Threshold value (input)	28
200BH	Count of abnormal readings (output)	06
200CH	Maximum temperature (output)	46

8085 Assembly Program

```

;=====
; Q2 : CHEMICAL PLANT - TEMPERATURE MONITORING
;=====
; DATA
; 2000H - 2009H : ten temperature readings
; 200AH : threshold value
; RESULTS
; 200BH : count of ABNORMAL readings ( > thr )
; 200CH : maximum temperature
;-----
LXI H,2000H ; HL -> first reading
MVI C,0AH ; C = reading counter (10)
MVI B,00H ; B = abnormal count = 0
MVI D,00H ; D = maximum temp = 0
LDA 200AH ; A = threshold
MOV E,A ; E = threshold (kept)

LOOP: MOV A,M ; A = current reading
      CMP E ; reading - threshold
      JC MAXCHK ; reading < thr -> normal
      JZ MAXCHK ; reading = thr -> normal
      INR B ; reading > thr -> ABNORMAL
MAXCHK: MOV A,M ; reload reading
      CMP D ; reading - current max
      JC NEXT ; reading < max -> skip
      MOV D,A ; else max = reading
NEXT: INX H ; next reading
      DCR C ; one reading done
      JNZ LOOP ; repeat for all 10

; ---- store results ----
MOV A,B
STA 200BH ; abnormal reading count
MOV A,D
STA 200CH ; maximum temperature
HLT ; stop

```

Sample Test Case (Verification)

Inputs: readings = 25, 30, 45, 50, 60, 35, 70, 20, 55, 65; threshold = 40 (28H).

--	--	--

Computer Architecture (CSA12) — 8085 ALP Solutions Page 4

200BH	06	Six readings exceed 40: 45, 50, 60, 70, 55, 65
200CH	46	Maximum = 70

Explanation

The same pointer/counter pattern is used (**HL**, **C** = 10). The threshold is loaded once into **E** so it need not be re-fetched every pass. The abnormal test hinges on both flags of `CMP E` (reading–threshold): a set **CY** means reading < threshold and a set **Z** means they are equal — both count as ‘normal’, so only a strictly greater reading (neither flag set) increments **B**. **D** tracks the peak via `CMP D`. The results are written to 200BH (count) and 200CH (maximum).

Marks of ten students are stored in memory. A student passes when the mark is greater than or equal to the stored pass mark. The program counts the passers, finds the highest mark, computes the total of all marks, and stores each result in its own location.

Algorithm

- Initialise: pointer → first mark, counter = 10, pass count = 0, highest = 0, 16-bit total = 0.
- Read a mark; add it to the 16-bit total, incrementing the high byte on carry.

- Compare the mark with the current highest; update if larger.
- Compare the mark with the pass mark; if mark $\geq$ pass mark, increment the pass count.
- Advance the pointer, decrement the counter, and repeat until all ten are processed.
- Store the pass count, the highest mark, and the total (low, high).

Memory Map

2000H – 2009H	Marks of ten students (input)	2D 4E 5A 21 3C 55 28 5F 37 46
200AH	Pass mark (input)	28
200BH	Number of students passed (output)	09
200CH	Highest mark (output)	5F
200DH / 200EH	Total marks low / high byte (output)	8B / 02

8085 Assembly Program


```

;=====
; Q3 : STUDENT MARKS - RESULT ANALYSIS
;=====
; DATA
; 2000H - 2009H : marks of ten students
; 200AH : pass mark
; RESULTS
; 200BH : number passed ( mark >= pass )
; 200CH : highest mark
; 200DH : total marks - LOW byte
; 200EH : total marks - HIGH byte
;-----
LXI H,2000H ; HL -> first mark
MVI B,0AH ; B = student counter (10)
MVI C,00H ; C = pass count = 0
MVI D,00H ; D = highest mark = 0
MVI E,00H ; E = total LOW byte = 0
XRA A ; A = 0
STA 200EH ; total HIGH byte = 0

LOOP: MOV A,M ; A = current mark
ADD E ; add to running total (low)
MOV E,A ; save low byte
JNC NOCRY ; no carry -> high byte same
LDA 200EH ; else bump total HIGH byte
INR A
STA 200EH
NOCRY: MOV A,M ; reload mark
CMP D ; mark - highest
JC CHKPAS ; mark < highest -> skip
MOV D,A ; else highest = mark
CHKPAS: LDA 200AH ; A = pass mark
CMP M ; pass mark - current mark
JC PASS ; pass<mark -> mark>pass -> PASS
JZ PASS ; equal -> PASS
JMP NEXT ; mark<pass -> fail
PASS: INR C ; count this pass
NEXT: INX H ; next mark
DCR B ; one student done
JNZ LOOP ; repeat for all 10

; ---- store results ----
MOV A,C
STA 200BH ; students passed
MOV A,D
STA 200CH ; highest mark
MOV A,E
STA 200DH ; total marks - LOW byte
HLT ; 200EH already holds HIGH

```

Sample Test Case (Verification)

Inputs: marks = 45, 78, 90, 33, 60, 85, 40, 95, 55, 70; pass mark = 40 (28H).

200BH	09	Nine pass (only 33 is below 40)
200CH	5F	Highest mark = 95
200DH : 200EH	8B : 02	Total = 028BH = 651

Explanation

HL is the pointer and **B** the counter (10). Ten marks can exceed 255, so the total is 16-bit: the low byte lives in **E** while the high byte is kept at 200EH and incremented only when `ADD E` sets **CY**. The pass test uses a neat trick — with the pass mark in **A**, `CMP M` computes `passmark - mark`, so a set **CY** (`passmark < mark`) or a set **Z** (equal) both mean the student passed (`mark ≥ passmark`); this avoids needing a spare register to hold the mark and the

pass mark at the same time. **D** tracks the highest mark. Results are written to 200BH (passers), 200CH (topper) and 200DH–200EH (total).